KRISHNAIK ACADEMY

AI SYSTEMS

LIVE COHORT | ADVANCED PROGRAM

AI System Design Bootcamp

Build, Deploy and Scale Production AI

V1.0



EVERY PROJECT, EVERY LAYER

- APPLICATION
- AI LAYER
- DATA
- INFRASTRUCTURE

MODERN AI SDLC | CODING AGENTS | ALWAYS-ON AGENTS | INFERENCE ENGINEERING | AI INFRASTRUCTURE | SYSTEM DESIGN | PRODUCTION DEPLOYMENT

BEFORE WE BEGIN

Everyone is building with AI. Very few are shipping it.

With today's coding agents, building an AI app takes an afternoon. Most of us have built one, maybe several. So be honest with yourself:

Have you made it an application?
Not a notebook or a demo, but a real product people can open and use.

Have you made it live?
Deployed on real infrastructure, with a URL, monitoring and a rollback plan.

Have you scaled it?
Beyond your laptop, beyond one GPU, beyond a handful of test users.

Have you scaled it to 10K users?
Fast for every user, without the bill or the latency exploding.

Have you earned money from it?
Subscriptions, payments and a product that someone values enough to pay for.

The questions an AI System Designer can answer

What really happens inside the GPU
when 10,000 people ask your model a question at the same time?

What does one AI request cost you,
and how would you cut that cost in half?

Could you draw the architecture
across the application, AI, data and infrastructure layers?

Who is watching your agents
while they work for you around the clock?

If you answered "not yet" to any of these, this bootcamp is built for you. We take you from building AI to designing, deploying and scaling AI systems.

Course Overview

This advanced bootcamp turns you into an **AI System Designer**: an engineer who can design, build, deploy and operate production AI end to end. You will learn how software is built in the AI era with spec-driven development and coding agents, build and govern always-on agents, and master inference engineering. You then apply it all across six production projects, from self-hosted and on-device LLMs to multimodal company intelligence, voice AI and a paid SaaS launch. Every project is deployed, monitored, secured and costed, and system design is taught through each one rather than as abstract theory.

Learning Objectives

- Build software the AI-native way with PRDs, spec-driven development (GitHub Spec Kit), context engineering and Agent Skills.
- Use Claude Code, Cursor and OpenAI Codex at an advanced level, including subagents, hooks, MCP and automated PR review.
- Build always-on agent fleets on sandboxed cloud computers and track them with an AgentOps control tower.
- Size GPUs, benchmark inference engines (vLLM, SGLang, TensorRT-LLM) and cut latency and cost with batching, caching and quantisation.
- Self-host and fine-tune open-source LLMs on company infrastructure with Terraform, and load test them against an SLO.
- Run LLMs on Android and iOS devices with a hybrid cloud fallback, and ship signed apps to testers.
- Design multimodal RAG at scale with permission-aware retrieval, agents, MCP and A2A.
- Connect AI to real business systems, including text-to-SQL analytics and self-hosted B2B deployments.
- Build low-latency voice agents on Indian telephony with Hindi and regional languages.
- Package and launch a paid SaaS: desktop apps, code signing, subscriptions, payments and admin dashboards.
- Secure, evaluate and observe AI systems with OWASP risk checks, guardrails, evals, tracing and cost control.
- Explain architecture decisions and business value to both technical and non-technical stakeholders.

Course Information

28	56	4	6
weeks	live sessions	core modules	production projects

Prerequisites

Working knowledge of **Python or JavaScript**, **Git** and the command line, plus a basic understanding of **LLMs, prompts and APIs** (familiarity with RAG and agents helps). You will need a laptop, a cloud account (AWS, GCP or Azure) with billing enabled for GPU labs, and an Android phone for the mobile project. A paid plan for at least one coding agent (Claude Code, Cursor or OpenAI Codex) is recommended.

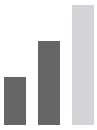
Approximately 6.5 months (28 weeks). Live classes run on Saturday and Sunday mornings, with recordings and starter repositories for every project. The course moves from the modern AI-native SDLC and coding agents, through inference engineering, into six production projects that are each deployed, monitored, secured and costed.

Estimated Time



6.5 months, 8 to 10 hrs/week*

Required Skill Level



Intermediate to Advanced

FORMAT
Live online, weekend mornings, with recordings

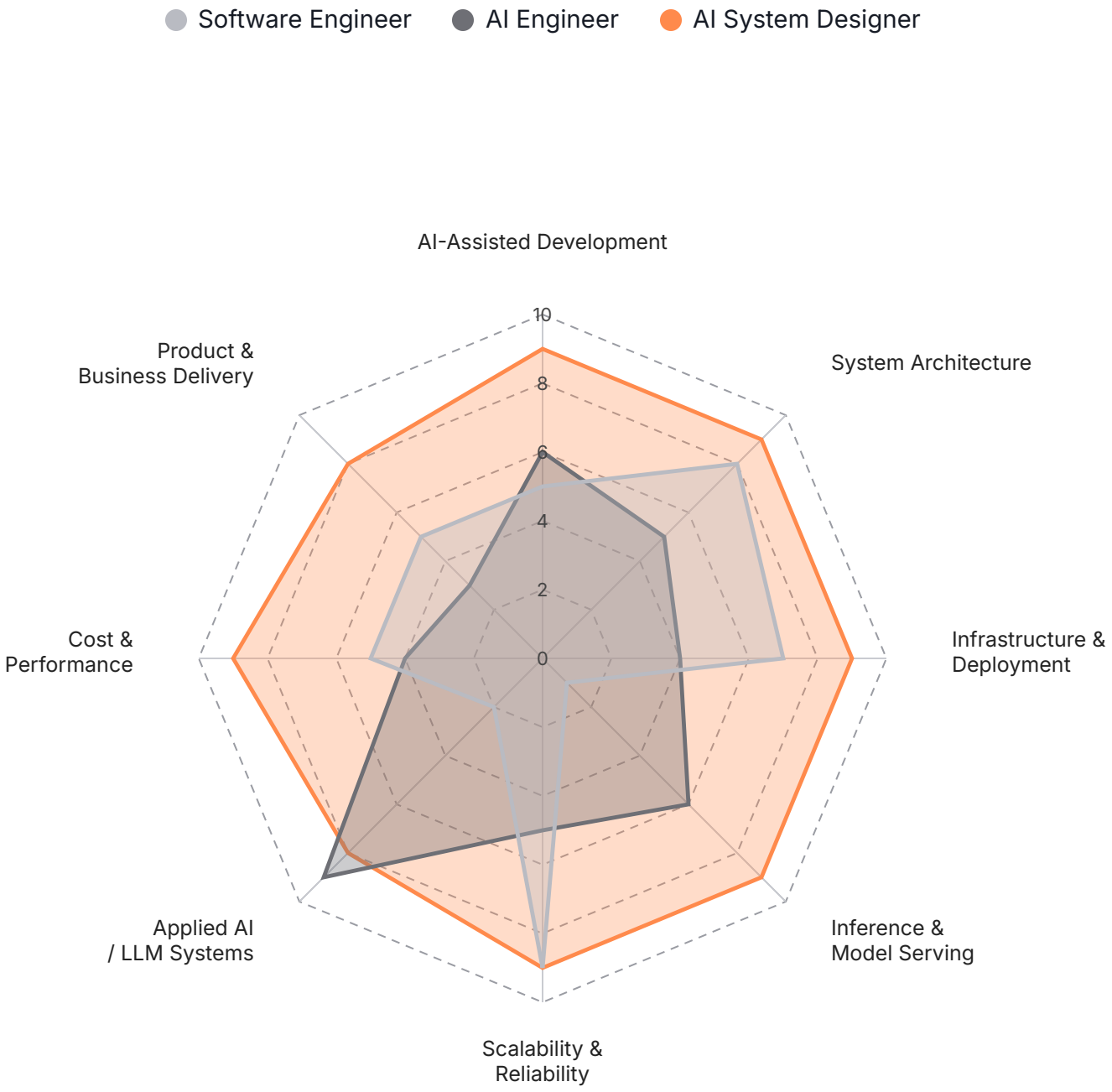
LEARNING MODEL
Project-assisted: concept, architecture, then build with AI coding agents

OUTCOME
A portfolio of deployed production AI systems

*Approximate: about 6 hours of live classes plus practice time each week, depending on prior experience.

Role Focus Matrix

SDE vs AI Engineer vs AI System Designer



Illustrative comparison of where each role focuses (0 to 10).

Krish Naik Academy Team



Krish Naik
Chief AI Engineer



Boktiar Ahmed Bappy
AI Engineer



Monal Kumar
Senior Data Scientist



Sunny Savita
GenAI Engineer



Mayank Aggrawal
Senior ML Engineer



Sourangshu Pal
Senior Data Scientist

Your Learning Journey

Four parts, ten modules and projects, 28 weeks. Each step builds on the one before it, from how software is built today to running AI in production.

PART 1: MODERN AI DEVELOPMENT AND AGENTS

Module 1	AI-Assisted Development: The Modern SDLC	2 weeks
Module 2	AI Coding Tools: Claude Code, Cursor, OpenAI Codex and More	2 weeks
Module 3	Always-On AI Agents and AgentOps: Build and Track Your Own Agent Fleet	3 weeks

PART 2: INFERENCE ENGINEERING

Module 4	Inference Engineering: Making LLMs Fast and Affordable	2 weeks

PART 3: AI INFRASTRUCTURE AND ENTERPRISE AI

Project 1	Self-Host and Fine-Tune an LLM on Company Infrastructure	3 weeks
Project 2	Run an LLM on Mobile Devices	3 weeks
Project 3	Multimodal Company Artefact Intelligence at Scale	4 weeks

PART 4: REAL-WORLD AI PRODUCTS (BUSINESS AND CONSUMER)

Project 4	Enterprise Lead Management System	3 weeks
Project 5	AI Receptionist System	3 weeks
Project 6	AI Voice Productivity SaaS: From Desktop App to Paid Product	3 weeks

How every project is built



Each project starts with an architecture diagram across four layers (application, AI, data and infrastructure), uses a starter repository, is built live with AI coding agents in small reviewable commits, and must be deployed, monitored, secured and costed before it is done.

Module 1 | 2 weeks | Part 1: Modern AI Development and Agents

AI-Assisted Development: The Modern SDLC

Software is no longer written line by line. Teams now define intent, turn it into specifications, and let AI agents implement while humans review and decide. This module rebuilds your development process for that reality, so everything you build later in the course is planned, traceable and safe to ship.

Topics	
How software is built now	SDLC evolution: waterfall, agile, DevOps, AI-native, AWS AI-DLC: Inception, Construction, Operations; short 'bolts' instead of sprints, DORA 2025: AI amplifies strong teams and exposes weak ones, New role of the developer: intent, review, decisions, Where AI-written code fails: security, 'almost right' logic
PRDs and spec-driven development	Writing PRDs agents can execute: problem, users, stories, acceptance criteria, Spec-driven development levels: spec-first, spec-anchored, spec-as-source, GitHub Spec Kit: constitution, specify, plan, tasks, implement, Kiro specs, OpenSpec for changes to existing code, BMAD agent personas, When specs are overkill
Context engineering and Agent Skills	CLAUDE.md, AGENTS.md and rules files; avoiding instruction bloat, Agent Skills: SKILL.md structure, scripts, references, progressive loading, MCP as the standard way to give agents tools and data, Options landscape: AWS, GCP, Azure; open and closed models; coding agents, Choosing tools for a team: cost, privacy, lock-in
Engineering discipline with agents	Git branching, small atomic commits, worktrees for parallel agents, Pull request review of AI-written code, Testing with AI: specs first, unit and end-to-end tests, Feedback sensors: linters, type checks, security and secret scanning, CI with GitHub Actions and AI review bots

WHAT YOU WILL BUILD

3 to 4 small mini projects (3 to 4 files each), each built from a PRD and a spec, with a clean commit history and CI checks.

ARCHITECTURE FOCUS

The AI-native delivery pipeline: PRD > spec > plan > tasks > code > tests > review > deploy, with human approval points.

Key tools GitHub Spec Kit | Kiro | OpenSpec | BMAD | AGENTS.md | Agent Skills | Git | GitHub Actions

Module 2 | 2 weeks | Part 1: Modern AI Development and Agents

AI Coding Tools: Claude Code, Cursor, OpenAI Codex and More

Every serious engineering team now works with coding agents. This module goes deep on Claude Code and covers Cursor, OpenAI Codex and the wider landscape, so you can pick the right tool for each job and run agents safely. The focus is on patterns that carry over as tools change every month.

Topics	
Claude Code I: core workflow	Setup, models and permission modes, Plan mode and CLAUDE.md, Slash commands and context management, Subagents and background tasks, Worktrees for parallel work
Claude Code II: extend and automate	Skills and plugins, Hooks, for example blocking destructive commands on production, Connecting MCP servers, Headless mode, Agent SDK and GitHub Actions, Built-in code review
Cursor and OpenAI Codex	Cursor: agents window, rules, parallel and cloud agents, best-of-n, Bugbot, Codex: CLI, IDE extension, cloud tasks, code review, AGENTS.md as the shared contract across tools, Comparing output quality, speed and cost
The wider landscape and choosing tools	GitHub Copilot coding agent and agentic workflows, Google Antigravity, Kiro, Devin, App builders: Lovable, Bolt, Replit, Open source: Cline, OpenCode, Aider, AI-generated infrastructure: Gemini Cloud Assist and Terraform, Decision matrix: terminal, IDE, cloud agent, app builder

WHAT YOU WILL BUILD

One app built with Claude Code, then extended with Cursor and Codex, plus your own custom skill, hook and automated PR review.

ARCHITECTURE FOCUS

How a coding agent works under the hood: model, tools, context, permissions and the feedback loop with tests.

Key tools Claude Code | Cursor | OpenAI Codex | GitHub Copilot | Google Antigravity | Kiro | Cline | OpenCode

Module 3 | 3 weeks | Part 1: Modern AI Development and Agents

Always-On AI Agents and AgentOps: Build and Track Your Own Agent Fleet

2026 brought always-on agents to the mainstream: SpaceXAI's Grok Bot, Meta's Muse and OpenAI's Dots each run on their own cloud computer, use apps on your behalf and ask for approval before sensitive actions. You build your own fleet of always-on agents and an Agent Control Tower that tracks every agent's work, cost, approvals and audit trail.

Topics	
The always-on agent era	Grok Bot, Meta Muse, OpenAI Dots: what they do and how they differ, What makes an agent always-on: persistent computer, memory, triggers, approvals, Reference architecture
Agent runtime	Sandboxed cloud computers and containers, Computer use and browser automation, Tools via MCP, Schedules, event triggers, long-running tasks, checkpoints
Memory and identity	Long-term memory and preferences, Scoped credentials, the agent as its own user, Secrets vault and least privilege
Multi-agent fleet	Specialist agents and shared threads, A2A coordination and task routing, Human approvals and reviewer agents
AgentOps Control Tower	Tracing with OpenTelemetry and Langfuse, Cost per agent and per task, success rates, Approvals queue and audit logs, Kill switch and alerts
Safety, evaluation and demo	OWASP Agentic Top 10, Red teaming agents, Task evaluations and governance policies, Fleet demo

WHAT YOU WILL BUILD

A team of always-on agents (Grok Bot, Muse and Dots style) running on cloud computers, plus an Agent Control Tower that tracks every agent's tasks, cost, approvals and audit trail.

ARCHITECTURE FOCUS

Triggers (chat, Slack, schedule, events) > orchestrator > agents in sandboxed cloud computers (tools via MCP, computer use, memory) > approval gate > Control Tower (traces, cost, audit, alerts).

Key tools Claude Agent SDK | OpenAI Agents SDK | LangGraph | MCP | A2A | E2B | Docker sandboxes | Playwright | Browser Use | Langfuse | OpenTelemetry

Module 4 | 2 weeks | Part 2: Inference Engineering

Inference Engineering: Making LLMs Fast and Affordable

Inference is where most of an AI product's speed and running cost are decided. This module explains what really happens when a model generates text, and how engineers cut latency and GPU cost with the right engine, batching, caching and quantisation. Projects 1 to 3 build directly on it.

Topics	
How LLM inference works	Tokens, prefill and decode, Why decode is memory-bound, KV cache and GPU memory maths, Choosing GPUs: L4, A10G, L40S, H100, Metrics: time to first token, inter-token latency, throughput, goodput, cost per million tokens
Inference engines and batching	vLLM, SGLang, TensorRT-LLM, llama.cpp: strengths and trade-offs, Continuous batching and PagedAttention, Prefix caching for chat and RAG, OpenAI-compatible serving APIs
Faster and cheaper	Quantisation: FP8, INT8, INT4 (AWQ, GPTQ, GGUF), Measuring quality loss after quantisation, Speculative decoding, Multi-LoRA serving, Structured outputs
Scaling inference	Tensor and pipeline parallelism, Disaggregated prefill and decode (llm-d, NVIDIA Dynamo), Autoscaling on queue depth, LLM gateway: routing, semantic cache, rate limits (LiteLLM), Self-host vs API break-even, Monitoring with Prometheus and Grafana

WHAT YOU WILL BUILD

A benchmark lab: the same open-source model served several ways (different engines, precisions and settings), with a report showing which setup gives the best speed per rupee.

ARCHITECTURE FOCUS

Request > gateway (auth, routing, cache) > scheduler and batcher > inference engine with KV cache on GPU > metrics. Shows exactly where latency and cost are won or lost.

Key tools vLLM | SGLang | TensorRT-LLM | llama.cpp | Ollama | LiteLLM | GuideLLM | Prometheus | Grafana

Project 1 | 3 weeks | Part 3: AI Infrastructure and Enterprise AI

Self-Host and Fine-Tune an LLM on Company Infrastructure

Companies want AI without sending their data outside. In two stages you first host an open-source small model on your own cloud and use it like an API, then fine-tune it on custom data, redeploy it and prove it can serve 10,000 users within a defined latency target.

Topics	
Architecture and infrastructure as code	Requirements for a private company LLM, App, AI and infra layer diagram, Choosing an SLM, Terraform: providers, state, modules, Reviewing AI-generated Terraform
Deploy and connect	vLLM in Docker on a cloud GPU, Gateway with auth, keys and rate limits, Connecting a chat app, Secrets management
Fine-tuning on custom data	Fine-tune vs RAG vs prompting, Dataset preparation and PII scrubbing, LoRA and QLoRA with Unsloth and TRL, Experiment tracking
Evaluate and redeploy	Evaluation: task metrics and LLM-as-judge, Alignment overview (DPO), Serving adapters and versioning models, Safe rollout and rollback
Scale to 10,000 users	Turning 10,000 users into peak concurrency and an SLO, Load testing with k6 and GuideLLM, Vertical vs horizontal scaling, Autoscaling options, Kubernetes overview
Operate it	Monitoring, alerts and logs, Cost dashboards, Self-host vs API comparison, Runbook and demo

WHAT YOU WILL BUILD

Stage 1: an open-source small model hosted on your own cloud and used like an API. Stage 2: the same model fine-tuned on custom data, redeployed and load tested for 10,000 users.

ARCHITECTURE FOCUS

App layer (chat app, API gateway, auth, rate limits) > AI layer (model server on GPU) > Infra layer (network, GPU servers, autoscaling, monitoring).

Key tools vLLM | Hugging Face | Unsloth | TRL | Terraform | AWS | GCP | Azure | Docker | k6 | GuideLLM | Grafana

Project 2 | 3 weeks | Part 3: AI Infrastructure and Enterprise AI

Run an LLM on Mobile Devices

On-device AI removes API bills, works offline and keeps user data private. You take a small model from a file to an app in real users' hands, tested on both Android and iPhone, with a cloud fallback for heavy requests.

Topics	
On-device AI architecture	Why on-device: cost, privacy, offline, Device limits: RAM, battery, thermal, Small model families and formats (GGUF, ExecuTorch, LiteRT), Hybrid design
Compressing models for phones	4-bit quantisation, Distillation overview, Measuring tokens per second, memory, battery
Build the Android app	Vibe-coding the app from a spec, Integrating react-native-executorch or llama.rn, Streaming UI, Downloading models on demand
iOS and hybrid routing	iOS build, Apple Foundation Models option, Cloud fallback API and routing logic, Offline storage and privacy
Monetise and harden	In-app subscriptions with RevenueCat, Crash reporting and analytics, Device testing, separate Android and iOS branches
Release	Signing and EAS Build, Play Console closed testing, TestFlight, Store listing, privacy labels, over-the-air updates

WHAT YOU WILL BUILD

A mobile AI app that runs a small model fully on the phone, with a cloud fallback, tested on Android and iPhone and released to testers.

ARCHITECTURE FOCUS

On-device AI layer inside the app, a cloud fallback API for heavy requests, and the routing logic that decides which one answers.

Key tools Gemma and other small models | llama.cpp | ExecuTorch | LiteRT-LM | React Native (Expo) | RevenueCat

Project 3 | 4 weeks | Part 3: AI Infrastructure and Enterprise AI

Multimodal Company Artefact Intelligence at Scale

Most company knowledge is locked in documents nobody can search: PDFs, slides, scans, images and recordings. You build a company brain that searches and reasons across all of them, runs efficiently on GPU infrastructure, respects access rights and is operated by agents.

Topics	
Use case and architecture	The enterprise knowledge problem, Multimodal RAG architecture, Infrastructure cost model, GPU sizing for embedding, re-ranking and LLM on one server
Ingestion pipeline	Parsing PDFs, slides and scans (Docling, OCR), Images, audio and video transcription, Chunking strategies, Metadata and access lists, Queues for incremental ingestion
Multimodal embeddings	Text vs visual retrieval, ColPali and ColQwen (late interaction), Serving embedding models on GPU, Batch embedding
Vector databases at scale	Qdrant and Milvus, Multi-vector indexes, Hybrid search (keywords plus vectors), Filtering, sharding and replication
Re-ranking and generation	Multimodal re-rankers, Context assembly and citations, Embedding, re-ranker and LLM on one GPU server, Latency budget
Permissions and agents	Permission-aware retrieval, Agents over the knowledge base with LangGraph, Building an MCP server for company knowledge
Multi-agent with A2A	Departmental agents, A2A agent cards and tasks, Human approval, Agent security (OWASP Agentic Top 10)
Evaluate and operate	Retrieval evaluation (recall, MRR, RAGAS), Red teaming and guardrails, Observability with Langfuse, Cost per query, Demo

WHAT YOU WILL BUILD

A company brain that searches and reasons across every file type, with permission-aware answers, agents on top, and an evaluation dashboard.

ARCHITECTURE FOCUS

Ingestion pipeline > multimodal embeddings > vector database > re-ranker > LLM > agents (MCP, A2A) > company app with access control.

Key tools Docling | ColPali | ColQwen | Qdrant | Milvus | vLLM | LangGraph | MCP | A2A | RAGAS | Langfuse

Project 4 | 3 weeks | Part 4: Real-World AI Products (Business and Consumer)

Enterprise Lead Management System

Every business needs leads. Based on a system running for a live client, you build an AI system that discovers, enriches, scores and contacts leads, lets managers query their data in plain English, and runs securely inside a company's own cloud.

Topics	
Business problem and architecture	Walkthrough of the live client system, App layer vs AI layer, Data model, Queues and workers
Discovery and enrichment	Apollo API, Web research agent (Tavily), Data quality and deduplication, Third-party rate limits
AI scoring and outreach	Lead scoring with LLMs plus rules, Personalised emails with human approval, Follow-up sequences, Deliverability and consent
Unified inbox and reply AI	Reply classification and intent detection, Auto-drafted responses, Meeting booking
Talk to your data	Text-to-SQL with schema grounding, Safety: read-only access, row limits, Dashboards for non-technical users
Self-host and secure	Docker and Terraform deployment, Role-based access and audit logs, DPDP Act basics, backups, Demo to a business audience

WHAT YOU WILL BUILD

A complete AI system for lead discovery, enrichment, scoring and outreach, based on a system running for a live client.

ARCHITECTURE FOCUS

Business app (CRM screens, database) + AI layer (enrichment agent, scoring, email drafting, natural-language queries) + queues + self-hosted infra.

Key tools Apollo API | LLM APIs | Tavily | PostgreSQL | Docker | Terraform | AWS | GCP

Project 5 | 3 weeks | Part 4: Real-World AI Products (Business and Consumer)

AI Receptionist System

Voice AI is one of the fastest-growing AI services. Based on a real clinic deployment, you build a receptionist that answers calls 24×7 in Hindi and English, books appointments, sends confirmations and reports on every call.

Topics	
Voice AI architecture	Pipeline vs speech-to-speech models, Latency budget, Turn-taking and interruptions, Managed (Vapi, Retell) vs open source (LiveKit, Pipecat)
Build the receptionist	Conversation design, Tools: calendar booking, FAQ RAG, Transfer to a human
India-ready deployment	Hindi and regional voices (Sarvam), Telephony: Exotel, Plivo; Twilio limits in India, TRAI number series, consent, AI disclosure
Integrations	WhatsApp and SMS confirmations, Reminders, CRM or Sheets sync, Outbound follow-ups
Scale and quality	Concurrent calls and worker scaling, Call recording and transcripts, Transcript evaluations, Analytics dashboard, cost per minute
Go live and sell	Health data under DPDP, HIPAA awareness, Pricing model for clients, Onboarding a clinic, Demo

WHAT YOU WILL BUILD

A voice AI receptionist that answers calls 24×7, books appointments and sends confirmations, based on a real clinic deployment.

ARCHITECTURE FOCUS

Phone line > voice pipeline (speech to text, LLM, text to speech) > tools (calendar, WhatsApp, CRM) > call analytics dashboard.

Key tools Vapi | Retell | LiveKit | Pipecat | Sarvam AI | Exotel | Plivo | WhatsApp Business API

Project 6 | 3 weeks | Part 4: Real-World AI Products (Business and Consumer)

AI Voice Productivity SaaS: From Desktop App to Paid Product

This is the complete consumer product journey. You build a desktop dictation app that runs speech recognition locally, then launch it as a paid SaaS with accounts, subscriptions, payments and an admin dashboard, so you know exactly how a real AI business takes money.

Topics	
Product and architecture	Desktop app plus cloud backend, Tauri vs Electron, Local speech models: Whisper, whisper.cpp, Moonshine; language support
Build the core	Global hotkeys and audio capture, Local transcription, LLM clean-up and formatting, Pasting into any app, OS permissions
Package and release	DMG and EXE builds, Code signing and macOS notarisation, Windows signing, Auto-updates
SaaS backend	Accounts and authentication, Database, Licence server, Usage tracking
Payments and subscriptions	Plans and pricing, Razorpay (India), Dodo Payments or Paddle (global, merchant of record), Webhooks and idempotency, GST invoices, failed payments
Launch	Admin dashboard: users, revenue, usage, Landing page, Analytics funnel, Launch strategy and demo

WHAT YOU WILL BUILD

A desktop app for Mac and Windows: press a hotkey, speak, and clean text appears in any app. Launched as a paid product with subscriptions.

ARCHITECTURE FOCUS

Desktop app (local AI, no server cost) + cloud backend (accounts, payment webhooks, licence server, admin dashboard).

Key tools Whisper | whisper.cpp | Moonshine | Tauri | Electron | Razorpay | Dodo Payments | Paddle

What You Walk Away With

A portfolio of production systems

- A spec-driven multi-tool app with custom skills, hooks and automated PR review
- An always-on agent fleet with an AgentOps control tower
- An inference benchmark lab with a speed-per-rupee report
- A private, fine-tuned LLM on cloud GPUs with a load test and cost report
- An on-device mobile AI app in testers' hands
- A multimodal company knowledge system with agents, MCP and A2A
- An AI lead management system with plain-English analytics
- A voice AI receptionist on a real Indian phone number
- A paid desktop SaaS with subscriptions and an admin dashboard

The production standard every project meets

Architecture diagram and spec	Clean Git history and CI	Tests and AI evaluations	Security and access control
Observability and alerts	Cost measured and controlled	Deployed with rollback	Documented and demonstrated

Roles this prepares you for

- AI Engineer / Senior AI Engineer
- AI System Designer / AI Solutions Architect
- Inference or ML Platform Engineer
- Forward Deployed AI Engineer
- AI Product Builder or Technical Founder

Design it. Build it. Ship it. Scale it.

Live weekend classes | Recordings | Starter repositories for every project